Name: Acala, Matt John C.
Section: CPE21S2

1. Define the following:

A. Graph: A group of dots (vertices) connected by lines (edges). Think of a map with cities and the roads between them.

B. Vertex: One dot in the graph. Like a single city on the map.

C. Nodes: Same as a vertex. It is just another name for it.

D. Weight: A number written on an edge, like the distance or toll fee of a road.

E. Path: A route from one vertex to another by following the edges, like going from city to city.

F. Directed graph: A graph where the edges have arrows, so you can only go one way. Like one-way streets.

G. Connected graph: A graph where you can get from any vertex to any other vertex. No city is cut off from the rest.

H. Directed cyclic: A directed graph where you can follow the arrows and end up back where you started, like one-way streets that go around in a loop. If there is no loop, it is acyclic.

I. Adjacency matrix: A table of rows and columns that shows which vertices are connected. You put the weight (or 1) if there is an edge and 0 if there is none.

J. Adjacency list: A list where each vertex has its own list of the vertices it points to. Like a contact list where each person has a list of who they can call.

2. Identify the parts of the following graph.

Vertices (nodes): A, B, C, D

Edges: A to B, A to C, C to B, D to A

Weights: A to B = 3, A to C = 2, C to B = 1, D to A = 4

Type of graph: Directed (it has arrows), weighted (it has numbers), and has no loop.

Start and end: D is the start because nothing points to it. B is the end because it points to nothing.

Example path: D to A to C to B. The total weight is 4 + 2 + 1 = 7.

3. Plot the adjacency matrix

Rows are where the arrow starts and columns are where it ends. 0 means there is no edge. For example, row A and column B is 3 because A points to B with a weight of 3.

	A	**B**	**C**	**D**
A	0	3	2	0
B	0	0	0	0
C	0	1	0	0
D	4	0	0	0

4.

```cpp
#include <iostream>

int main() {
    char name[4] = {'A', 'B', 'C', 'D'};
    int graph[4][4] = {0};        // 0 means no edge

    graph[0][1] = 3;              // A -> B, weight 3
    graph[0][2] = 2;              // A -> C, weight 2
    graph[2][1] = 1;              // C -> B, weight 1
    graph[3][0] = 4;              // D -> A, weight 4

    std::cout << "Adjacency Matrix:\n  A B C D\n";
    for (int i = 0; i < 4; i++) {
        std::cout << name[i] << " ";
        for (int j = 0; j < 4; j++) {
            std::cout << graph[i][j] << " ";
        }
        std::cout << "\n";
    }

    std::cout << "\nEdges:\n";
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (graph[i][j] != 0) {
                std::cout << name[i] << " -> " << name[j] << " (" << graph[i][j] << ")\n";
            }
        }
    }
    return 0;
}

```

```
Adjacency Matrix:
  A B C D
A 0 3 2 0
B 0 0 0 0
C 0 1 0 0
D 4 0 0 0

Edges:
A -> B (3)
A -> C (2)
C -> B (1)
D -> A (4)
```

5. Differentiate BFS vs DFS types of search.

BFS (Breadth-First Search): Checks everything close to the start first, then moves outward level by level. Like the ripples when you drop a stone in water. It uses a queue.

DFS (Depth-First Search): Picks one path and goes as deep as it can until it hits a dead end, then goes back and tries another. Like walking through a maze. It uses a stack (or recursion).

Main difference: BFS goes wide first, DFS goes deep first.

When to use: BFS is good for finding the shortest path (fewest edges). DFS is good for checking loops and solving mazes.